REFERENCE GUIDE

Next.js Fundamentals

Core concepts with practical code examples

01

App Router Architecture

The foundation of modern Next.js

Next.js 13+ introduced the **App Router** using the `app/` directory. It replaces the older `pages/` directory with a more powerful model built on React Server Components.

FEATURE	PAGES ROUTER (`pages/`)	APP ROUTER (`app/`)
Default component type	Client Components	Server Components
Data fetching	`getServerSideProps`, `getStaticProps`	`async` components + `fetch`
Layouts	Manual via `_app.tsx`	Nested `layout.tsx` files
Streaming	Not built-in	Native Suspense streaming

A minimal project structure:

```
my-app/
  app/
    layout.tsx        ← root layout (required)
    page.tsx          ← home page at /
    about/
      page.tsx        ← /about
    blog/
      [slug]/
        page.tsx      ← /blog/hello-world
  public/             ← static files
  next.config.js
```

02

File-Based Routing

Folders become URL segments, special files control behavior

FILE	PURPOSE
`page.tsx`	Makes a route publicly accessible
`layout.tsx`	Shared UI wrapper — persists across navigations
`loading.tsx`	Loading UI (wraps page in Suspense)
`error.tsx`	Error boundary for the segment
`not-found.tsx`	UI shown when `notFound()` is called
`route.ts`	API endpoint (GET, POST, etc.)

Dynamic Routes

```
// Single dynamic segment
app/blog/[slug]/page.tsx        → /blog/hello-world

// Catch-all segments
app/docs/[...slug]/page.tsx     → /docs/getting-started/install

// Optional catch-all
app/docs/[[...slug]]/page.tsx   → /docs OR /docs/a/b/c
```

Route Groups

Folders wrapped in parentheses organize code without affecting the URL:

```
app/
  (marketing)/
    about/page.tsx       → /about
    pricing/page.tsx     → /pricing
  (app)/
    layout.tsx           → shared dashboard layout
    settings/page.tsx    → /settings
```

Parallel Routes

Named slots let you render multiple pages in the same layout simultaneously:

```
app/
  layout.tsx              → receives {children, analytics, team}
  page.tsx
  @analytics/page.tsx     → rendered alongside the main page
  @team/page.tsx
```

```tsx
// app/layout.tsx
export default function Layout({ children, analytics, team }) {
  return (
    <div>
      {children}
      <aside>{analytics}</aside>
      <aside>{team}</aside>
    </div>
  );
}
```

Intercepting Routes

Show a route in a modal while preserving the background context (like Instagram photo modals):

```
app/
  feed/page.tsx
  feed/(.)photo/[id]/page.tsx   → intercepts /photo/[id] from feed
  photo/[id]/page.tsx             → full page on direct visit
```

The `(.)` prefix matches the same level, `(..)` matches one level up, `(...)` matches the root.

03

Layouts & Templates

Shared UI that nests automatically

A `layout.tsx` wraps all pages in its segment and below. Layouts **preserve state** across navigations — they don't remount.

```
// app/layout.tsx — the root layout (required)
export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        <nav>Site Navigation</nav>
        {children}
      </body>
    </html>
  );
}

// app/blog/layout.tsx — nested layout for /blog/*
export default function BlogLayout({ children }) {
  return (
    <div className="blog-grid">
      <Sidebar />
      <main>{children}</main>
    </div>
  );
}
```

Templates

A `template.tsx` works like a layout but **remounts on every navigation** — useful for enter/exit animations or per-page analytics logging.

```
// app/template.tsx
export default function Template({ children }) {
  return <div className="page-transition">{children}</div>;
}
```

04

Server & Client Components

The mental model at the heart of the App Router

All components in the App Router are **Server Components by default**. They run on the server, can access databases and APIs directly, and send zero JavaScript to the browser.

```
// Server Component (default — no directive needed)
import { db } from '@/lib/db';

export default async function UserList() {
  const users = await db.user.findMany();
  return (
    <ul>
      {users.map(u => <li key={u.id}>{u.name}</li>)}
    </ul>
  );
}
```

Client Components

Add `'use client'` at the top when you need interactivity, hooks, or browser APIs:

```
'use client';
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(c => c + 1)}>
      Count: {count}
    </button>
  );
}
```

Composition: The "Donut" Pattern

A Client Component can't *import* a Server Component. But a Server Component can *pass* a Server Component as `children` to a Client Component:

```tsx
// ServerPage.tsx (Server Component)
import { ClientWrapper } from './client-wrapper';
import { ServerData } from './server-data';

export default function Page() {
  return (
    <ClientWrapper>         {/* has onClick, state */}
      <ServerData />          {/* fetches data on server */}
    </ClientWrapper>
  );
}
```

> **Rule of thumb:** Keep the boundary as low as possible. Only wrap the interactive leaf — not the whole page — in `'use client'`.

05

Rendering Strategies

SSG, SSR, ISR, and Partial Prerendering

Static (SSG)

Pages rendered at build time. Use `generateStaticParams` for dynamic routes:

```
export async function generateStaticParams() {
  const posts = await getPosts();
  return posts.map(p => ({ slug: p.slug }));
}

export default async function Post({ params }) {
  const post = await getPost(params.slug);
  return <article>{post.content}</article>;
}
```

Dynamic (SSR)

Rendered on every request. Opt in by using `cache: 'no-store'` or the segment config:

```
export const dynamic = 'force-dynamic';

export default async function Dashboard() {
  const data = await fetch('https://api.example.com/live', {
    cache: 'no-store'
  });
  return <Stats data={data} />;
}
```

Incremental Static Regeneration (ISR)

Static pages that refresh in the background after a set interval:

```
const res = await fetch('https://api.example.com/products', {
  next: { revalidate: 60 }  // re-generate every 60 seconds
});
```

Partial Prerendering (PPR)

Next.js 14+ experimental feature. Combines a static shell (served instantly from CDN) with dynamic content that streams in:

```
export const experimental_ppr = true;

export default function Page() {
  return (
    <main>
      <StaticHero />                {/* served instantly */}
      <Suspense fallback={<Skeleton />}>
        <PersonalizedFeed />        {/* streams in */}
      </Suspense>
    </main>
  );
}
```

06

Data Fetching

Async components replace getServerSideProps entirely

Server Components can be `async`. Fetch data directly — Next.js deduplicates identical requests in the same render pass automatically.

```
export default async function ProductPage({ params }) {
  const product = await fetch(
    `https://api.store.com/products/${params.id}`
  ).then(r => r.json());

  return (
    <div>
      <h1>{product.name}</h1>
      <p>{product.description}</p>
    </div>
  );
}
```

Parallel Fetching

Use `Promise.all` to avoid sequential waterfalls:

```
export default async function Page() {
  const [user, posts, stats] = await Promise.all([
    getUser(),
    getPosts(),
    getStats(),
  ]);

  return (
    <div>
      <Profile user={user} />
      <PostList posts={posts} />
      <Dashboard stats={stats} />
    </div>
  );
}
```

Using an ORM Directly

Since Server Components run on the server, you can query your database without an API layer:

```
import { prisma } from '@/lib/prisma';

export default async function Users() {
  const users = await prisma.user.findMany({
    orderBy: { createdAt: 'desc' },
    take: 10,
  });
  return <UserTable users={users} />;
}
```

07

Server Actions

Mutate data from the client without building an API

Server Actions are async functions that run on the server. Call them from forms or event handlers. They're marked with `'use server'`.

```
// app/actions.ts
'use server';

import { revalidatePath } from 'next/cache';
import { prisma } from '@/lib/prisma';

export async function createPost(formData: FormData) {
  const title = formData.get('title') as string;
  const body = formData.get('body') as string;

  await prisma.post.create({ data: { title, body } });
  revalidatePath('/posts');
}
```

Using in a Form

```
import { createPost } from './actions';

export default function NewPost() {
  return (
    <form action={createPost}>
      <input name="title" placeholder="Title" required />
      <textarea name="body" placeholder="Content" />
      <button type="submit">Publish</button>
    </form>
  );
}
```

With Pending State (React 19)

```
'use client';
import { useActionState } from 'react';
import { createPost } from './actions';

export function PostForm() {
  const [state, action, isPending] = useActionState(createPost, null);

  return (
    <form action={action}>
      <input name="title" disabled={isPending} />
      <button>{isPending ? 'Publishing...' : 'Publish'}</button>
      {state?.error && <p>{state.error}</p>}
    </form>
  );
}
```

> **Progressive enhancement:** Forms using Server Actions work even when JavaScript is disabled — the browser submits a standard POST request.

08

Route Handlers (APIs)

Build REST endpoints alongside your pages

Create a `route.ts` file and export functions named after HTTP methods:

```
// app/api/users/route.ts
import { NextResponse } from 'next/server';

export async function GET() {
  const users = await db.user.findMany();
  return NextResponse.json(users);
}

export async function POST(request: Request) {
  const body = await request.json();
  const user = await db.user.create({ data: body });
  return NextResponse.json(user, { status: 201 });
}
```

Dynamic API Routes

```
// app/api/users/[id]/route.ts
export async function GET(
  request: Request,
  { params }: { params: { id: string } }
) {
  const user = await db.user.findUnique({
    where: { id: params.id }
  });

  if (!user) {
    return NextResponse.json(
      { error: 'Not found' }, { status: 404 }
    );
  }
  return NextResponse.json(user);
}
```

Route Handlers support all standard HTTP methods: `GET`, `POST`, `PUT`, `PATCH`, `DELETE`, `HEAD`, and `OPTIONS`.

09

Middleware

Intercept every request before it reaches a route

Create a `middleware.ts` at the project root. It runs on the **Edge Runtime** before every matched request.

```
// middleware.ts
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
  // Redirect unauthenticated users
  const token = request.cookies.get('session');

  if (!token && request.nextUrl.pathname.startsWith('/dashboard')) {
    return NextResponse.redirect(
      new URL('/login', request.url)
    );
  }

  // Continue with custom header
  const response = NextResponse.next();
  response.headers.set('x-pathname', request.nextUrl.pathname);
  return response;
}

// Only run on these paths
export const config = {
  matcher: ['/dashboard/:path*', '/api/:path*'],
};
```

Common uses: authentication, redirects, A/B testing, geolocation routing, rate limiting, and adding security headers.

10

Caching & Revalidation

Four caching layers working together

LAYER	WHAT	WHERE	OPT OUT
Request Memoization	Dedupes identical `fetch` calls per render	Server	`AbortController`
Data Cache	`fetch` responses across requests	Server	`cache: 'no-store'`
Full Route Cache	Pre-rendered HTML + RSC payload	Server	`dynamic = 'force-dynamic'`
Router Cache	RSC payload in the browser	Client	`router.refresh()`

Revalidation

```
// Time-based — refresh in background after 60 seconds
fetch(url, { next: { revalidate: 60 } });

// On-demand — trigger from a Server Action or Route Handler
import { revalidatePath, revalidateTag } from 'next/cache';

revalidatePath('/blog');       // purge an entire route
revalidateTag('posts');       // purge by tag

// Tag a fetch for on-demand revalidation
fetch(url, { next: { tags: ['posts'] } });
```

11

Loading & Error States

Declarative UI for pending and failed states

Loading UI

A `loading.tsx` file automatically wraps the page in a `Suspense` boundary:

```
// app/dashboard/loading.tsx
export default function Loading() {
  return (
    <div className="skeleton">
      <div className="skeleton-header" />
      <div className="skeleton-body" />
    </div>
  );
}
```

Error Handling

An `error.tsx` file catches runtime errors and provides a recovery mechanism:

```
'use client';  // must be a Client Component

export default function Error({
  error,
  reset,
}: {
  error: Error;
  reset: () => void;
}) {
  return (
    <div>
      <h2>Something went wrong</h2>
      <p>{error.message}</p>
      <button onClick={() => reset()}>Try again</button>
    </div>
  );
}
```

Not Found

```tsx
// app/blog/[slug]/page.tsx
import { notFound } from 'next/navigation';

export default async function Post({ params }) {
  const post = await getPost(params.slug);
  if (!post) notFound();  // renders not-found.tsx
  return <article>{post.content}</article>;
}
```

Image & Font Optimization

Built-in performance without configuration

next/image

```
import Image from 'next/image';
import heroImg from './hero.jpg';  // static import

export default function Hero() {
  return (
    <Image
      src={heroImg}
      alt="Hero"
      priority              {/* preload for LCP */}
      placeholder="blur"    {/* blur-up while loading */}
      sizes="(max-width: 768px) 100vw, 50vw"
    />
  );
}
```

Automatic benefits: WebP/AVIF conversion, responsive `srcSet`, lazy loading, layout shift prevention.

next/font

Fonts are downloaded at build time and self-hosted — no external requests, no layout shift:

```
import { Inter, Merriweather } from 'next/font/google';

const inter = Inter({
  subsets: ['latin'],
  variable: '--font-sans',
});
const merriweather = Merriweather({
  subsets: ['latin'],
  weight: ['400', '700'],
  variable: '--font-serif',
});

export default function RootLayout({ children }) {
  return (
    <html className={`${inter.variable} ${merriweather.variable}`}>
      <body>{children}</body>
    </html>
  );
}
```

13

Metadata & SEO

Static and dynamic metadata in every route

Static Metadata

```
// app/layout.tsx or any page.tsx
export const metadata = {
  title: 'My App',
  description: 'Built with Next.js',
  openGraph: {
    title: 'My App',
    images: ['/og.png'],
  },
};
```

Dynamic Metadata

```
export async function generateMetadata({ params }) {
  const product = await getProduct(params.id);
  return {
    title: product.name,
    description: product.summary,
    openGraph: { images: [product.image] },
  };
}
```

Special SEO Files

- `sitemap.ts` — generate a dynamic XML sitemap
- `robots.ts` — generate robots.txt programmatically
- `opengraph-image.tsx` — generate OG images using JSX and `ImageResponse`

```ts
// app/sitemap.ts
export default async function sitemap() {
  const posts = await getPosts();
  return posts.map(post => ({
    url: `https://example.com/blog/${post.slug}`,
    lastModified: post.updatedAt,
  }));
}
```

14

Streaming with Suspense

Show content progressively as data arrives

Wrap slow components in `<Suspense>` to stream them independently. The page shell loads instantly while each boundary resolves on its own timeline.

```
import { Suspense } from 'react';

export default function Dashboard() {
  return (
    <main>
      <h1>Dashboard</h1>

      <Suspense fallback={<ChartSkeleton />}>
        <RevenueChart />          {/* streams when ready */}
      </Suspense>

      <Suspense fallback={<TableSkeleton />}>
        <RecentOrders />          {/* independent stream */}
      </Suspense>

      <Suspense fallback={<ListSkeleton />}>
        <Notifications />         {/* another stream */}
      </Suspense>
    </main>
  );
}
```

Each `<Suspense>` boundary creates an independent streaming chunk. The browser receives the static shell first, then each section fills in as its data resolves — no full-page loading spinner needed.

> **How it works:** Next.js sends an initial HTML response with the static parts and placeholder fallbacks. As each Server Component finishes rendering, its HTML is streamed to the browser and swapped in using React's hydration. Users see content progressively rather than waiting for the slowest query.